CLOUD-BASED DIGITAL CERTIFICATE VERIFICATION & SECURE STORAGE SYSTEM

PROBLEM STATEMENT

Digital certificates may be lost, modified, replaced, or stored without adequate protection. Manual verification can also be time-consuming, especially when many certificates must be managed. Traditional methods may require physical documents or direct communication with the issuing institution, and digital files can also be accidentally modified or replaced.

Therefore, a system is required to store certificate files securely, maintain their associated information, and detect changes to the stored files. The system should provide an easy-to-use interface for uploading and verifying certificates while reducing the need for manual file comparison.

ABSTRACT

The Cloud-Based Digital Certificate Verification & Secure Storage System is a web-based application developed to securely store digital certificates and verify whether their contents have been modified. The system uses cloud storage to maintain certificate files and cryptographic hashing to check their integrity.

The application is developed using Python and the Flask web framework. It allows users to upload certificates in formats such as PDF, DOCX, PNG, JPG, and JPEG. After uploading, the system extracts available certificate details, including the student's name, certificate ID, course name, issue date, and institution.

The uploaded certificate is stored in a private Amazon Simple Storage Service (Amazon S3) bucket. Server-side encryption using AES-256 helps protect the stored files. The application calculates a SHA-256 hash of the certificate and saves the expected hash along with the certificate's metadata in an SQLite database.

When a certificate is verified, the system retrieves the stored file, calculates its current SHA-256 hash, and compares it with the original hash. If both hashes match, the certificate passes the integrity check. If they differ, the system reports an integrity mismatch, indicating that the stored file may have been modified or replaced. This project demonstrates the use of cloud

computing, secure file storage, document processing, and cryptographic integrity verification in a practical application.

INTRODUCTION

Digital certificates are widely used by educational institutions, training centres, and organizations to confirm the completion of courses, training programs, and other achievements. As certificates are increasingly shared and stored electronically, ensuring their security and integrity has become important.

The proposed system provides a centralized web application for uploading certificates, storing them securely in the cloud, extracting relevant information, and checking whether the stored files have changed. By combining Flask, Amazon S3, SQLite, and SHA-256 hashing, the system offers a practical approach to certificate storage and integrity checking.

Modern certificate management systems require web applications capable of:

- Uploading certificates in multiple document and image formats
- Extracting certificate details automatically
- Storing certificate files securely in the cloud
- Protecting stored files using encryption
- Generating cryptographic hashes for integrity checking
- Detecting modified or replaced certificate files
- Maintaining certificate metadata and verification records

OBJECTIVES OF THE PROJECT

The main objectives of the proposed system are:

- To develop a web-based application for digital certificate management.
- To allow users to upload certificates in multiple document and image formats.
- To extract available certificate details automatically.
- To store certificate files in a private Amazon S3 bucket.
- To protect stored files using server-side encryption.
- To generate SHA-256 hashes for certificate integrity checking.
- To detect whether a stored certificate has been modified or replaced.

- To maintain certificate metadata and verification records.
- To provide a dashboard for viewing certificate and verification information.

PROPOSED SYSTEM

The proposed system is a web application designed for secure certificate storage, automatic detail extraction, and integrity verification using cloud storage and cryptographic hashing.

The system architecture consists of:

- User Interface Layer
- Flask Application Layer
- Document Extraction Layer
- Cloud Storage Layer (Amazon S3)
- Database Layer (SQLite)
- Integrity Verification Layer

FEATURES OF PROPOSED SYSTEM

CERTIFICATE MANAGEMENT FEATURES

- Certificate Upload (DOCX, PDF, PNG, JPG, JPEG)
- File Validation and Size Limit (12 MB)
- Centralized Certificate Storage
- Certificate Metadata Management
- Certificate Record Retrieval
- Dashboard for Certificate Information

DOCUMENT EXTRACTION FEATURES

- Student Name Extraction
- Certificate ID Extraction
- Course Name Extraction
- Issue Date Extraction
- Institution Name Extraction
- Text Extraction from DOCX using python-docx

- Text Extraction from PDF using pdfplumber
- OCR for Scanned Documents and Images using Tesseract

INTEGRITY VERIFICATION MODULE

- SHA-256 Hash Generation
- Expected Hash Storage in Database
- Stored File Retrieval from Amazon S3
- Current Hash Calculation
- Hash Comparison
- Integrity Verified / Integrity Mismatch Result Display
- Verification History Tracking

SECURITY FEATURES

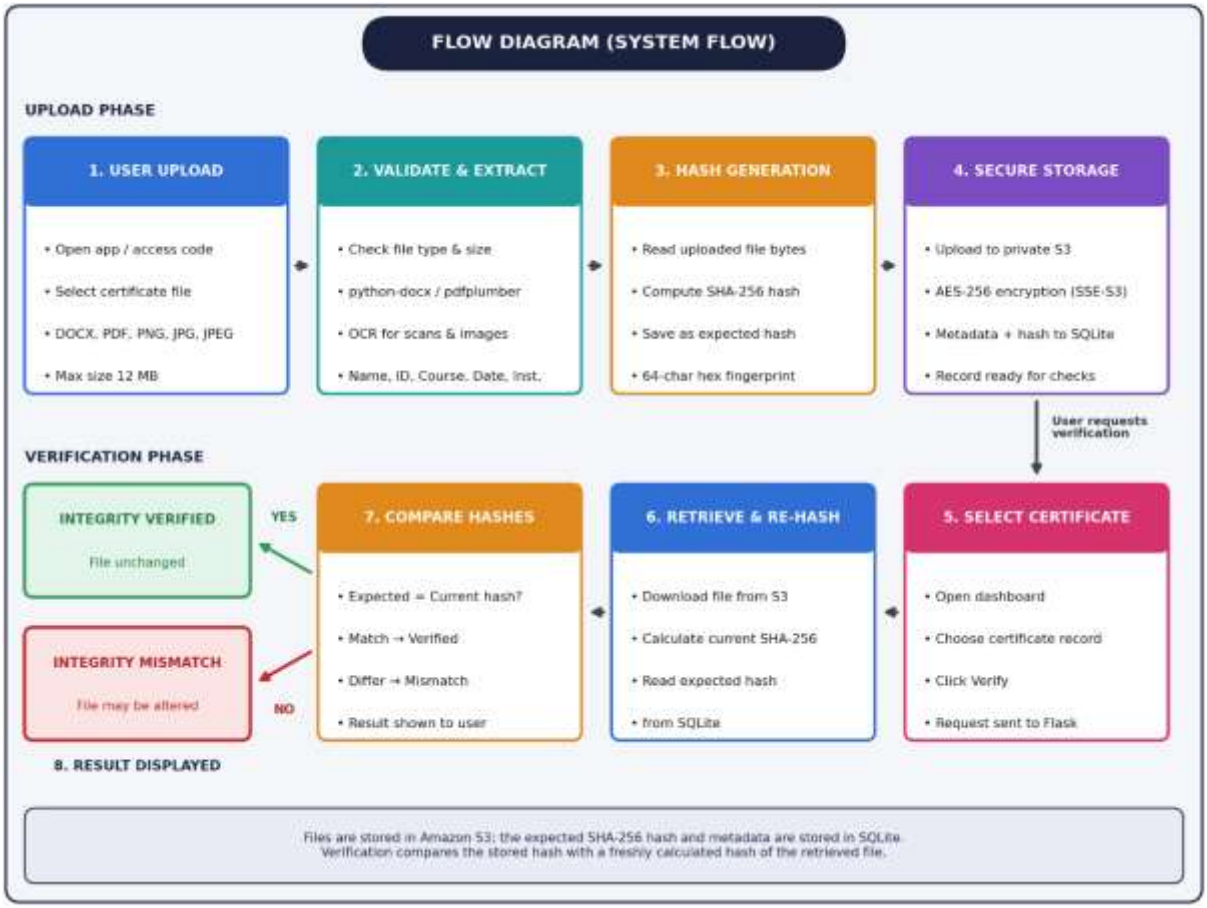
- Private Amazon S3 Bucket with Public Access Blocked
- AES-256 Server-Side Encryption (SSE-S3)
- SHA-256 Cryptographic Hashing
- AWS Credentials through Environment Variables
- Optional Demo Access Code
- File Type and Size Validation
- Error Handling

SUSTAINABLE DEVELOPMENT GOALS (SDGs)

The project supports the following SDGs:

- SDG 4: Quality Education
- SDG 9: Industry, Innovation, and Infrastructure
- SDG 16: Peace, Justice, and Strong Institutions

SYSTEM ARCHITECTURE & FLOW DIAGRAM



System Components

- **User Interface:** Allows users to upload certificates, view extracted details, and initiate verification.
- **Flask Application:** Receives user requests, validates uploaded files, processes documents, and coordinates storage and verification.
- **Document Extraction Module:** Extracts text from DOCX and PDF documents. OCR is used for supported scanned documents and images.
- **Amazon S3 Storage:** Stores certificate files in a private cloud bucket with server-side encryption enabled.
- **SQLite Database:** Stores certificate metadata, the expected SHA-256 hash, and relevant verification records.

- **Integrity Verification Module:** Downloads the selected certificate from S3, calculates its current hash, and compares it with the saved expected hash.

System Workflow

User uploads certificate → Flask validates the file → Text and certificate details are extracted → SHA-256 hash is generated → File is stored in private Amazon S3 → Metadata and expected hash are saved in SQLite → User requests verification → Stored file is downloaded and hashed again → Hashes are compared → Result is displayed.

TOOLS AND TECHNOLOGIES

Technology	Purpose
Python	Main programming language
Flask	Develops the web application and handles requests
HTML and CSS	Build the user interface
Amazon S3	Stores certificate files in the cloud
AWS SDK for Python (Boto3)	Connects the Flask application to Amazon S3
SQLite	Stores certificate metadata and expected hashes
SHA-256	Generates a digital hash for integrity verification
AES-256 SSE-S3	Protects stored S3 objects at rest
python-docx	Extracts text from DOCX documents
pdfplumber	Extracts text from text-based PDF files
Tesseract OCR and pytesseract	Extract text from scanned documents and images
Pillow	Supports image processing

WORKING OF THE SYSTEM

Step 1: Accessing the Application

The user opens the web application and, if the demo access-code feature is enabled, enters the configured demo access code to proceed.

Step 2: Uploading a Certificate

The user selects a certificate file and uploads it. The application supports DOCX, PDF, PNG, JPG, and JPEG formats, subject to the configured file-size limit of 12 MB.

Step 3: Extracting Certificate Details

The system attempts to identify the student name, certificate ID, course name, issue date, and institution name. For DOCX files, text is extracted from paragraphs and tables. For PDFs, text extraction is performed using pdfplumber. OCR using Tesseract is used for supported scanned PDFs and images. If a field cannot be identified, the system may display “Not detected.”

Step 4: Generating the SHA-256 Hash

The application calculates a SHA-256 hash from the uploaded file's bytes. This hash acts as a digital fingerprint of the file's contents. Even a small change in the file can produce a different hash. The expected hash is saved in the database for future comparison.

Step 5: Secure Cloud Storage

The certificate file is uploaded to a private Amazon S3 bucket. Public access is blocked, and server-side encryption using AES-256 is enabled for stored objects. The application uses AWS credentials configured through environment variables to communicate with S3.

Step 6: Saving Metadata

The application saves the certificate's available details, storage reference, expected SHA-256 hash, and other relevant metadata in SQLite. The database helps the application locate the correct certificate record during verification.

Step 7: Certificate Verification

When the user selects a certificate for verification, the application retrieves the corresponding file from Amazon S3 and calculates its SHA-256 hash again. The newly calculated hash is compared with the expected hash saved during upload.

Step 8: Displaying the Result

The application displays Integrity Verified if the current file hash matches the expected hash, or Integrity Mismatch if it differs, indicating that the file may have been modified or replaced. The dashboard can also display certificate totals and verification information.

SHA-256 INTEGRITY VERIFICATION

SHA-256 is a cryptographic hash function that converts input data into a fixed-length 256-bit hash, normally represented as a 64-character hexadecimal string. The application generates a hash for the certificate at upload time and saves it as the expected hash. During verification, it generates a new hash from the file retrieved from S3.

Verification Rule

- Expected hash = Current hash → Integrity Verified
- Expected hash ≠ Current hash → Integrity Mismatch

The hash comparison checks whether the retrieved file's bytes match those originally recorded by the application. It does not independently prove that the certificate was genuinely issued by the institution or that its original contents were truthful.

MODULES OF THE SYSTEM

Access Control Module

Handles the optional demo access code to restrict access to the demonstration interface.

Certificate Upload Module

Allows users to upload DOCX, PDF, PNG, JPG, and JPEG files and validates file type and size.

Document Extraction Module

Extracts certificate details using python-docx, pdfplumber, and Tesseract OCR.

Hash Generation Module

Calculates the SHA-256 hash of the uploaded certificate.

Cloud Storage Module

Stores certificate files in a private, encrypted Amazon S3 bucket using Boto3.

Database Management Module

Stores certificate metadata, expected hashes, and verification records in SQLite.

Integrity Verification Module

Retrieves the stored file, recalculates its hash, and compares it with the expected hash.

Dashboard Module

Displays certificate totals, certificate details, and verification information.

DATABASE DESCRIPTION

The SQLite database maintains the information required to manage and verify certificates. Depending on the application's database schema, records may include:

- Certificate record ID
- Student name
- Certificate ID
- Course name
- Issue date
- Institution name
- Original filename
- S3 object key or storage reference
- Expected SHA-256 hash
- Upload timestamp
- Verification history or status

The database stores metadata and the expected hash, while the certificate file itself is stored in Amazon S3.

SECURITY FEATURES IN DETAIL

- **Private Cloud Storage:** The S3 bucket is configured to block public access, helping prevent unauthorized public access to certificate files.
- **Server-Side Encryption:** Amazon S3 server-side encryption with Amazon S3-managed keys (SSE-S3) uses AES-256 encryption to protect objects at rest.
- **SHA-256 Hashing:** The expected hash is stored in the database and compared with the hash of the retrieved file to detect changes.

- **Environment Variables:** AWS credentials and application configuration are supplied through environment variables instead of being written directly into source code. Secret configuration files must not be uploaded to a public GitHub repository.
- **Demo Access Code:** An optional shared demo code acts as a basic access gate. It is not a replacement for individual user accounts, role-based authorization, or production-grade authentication.

ADVANTAGES

- Provides centralized storage for digital certificates.
- Reduces the need for manual file comparison.
- Supports multiple document and image formats.
- Automatically extracts available certificate details.
- Uses cloud storage for certificate files.
- Protects stored objects with server-side encryption.
- Detects changes to stored file contents through hash comparison.
- Maintains certificate metadata for later retrieval.
- Demonstrates the practical use of cloud computing and cryptographic techniques.

LIMITATIONS

- Text extraction may fail when documents are unclear, poorly scanned, or formatted unusually.
- OCR may produce incorrect or incomplete text.
- SHA-256 comparison detects changes relative to the saved hash but does not prove that the certificate is authentic.
- The expected hash and database must themselves be protected from unauthorized changes.
- The demo access code is not a complete user authentication system.
- The application depends on correct AWS configuration and network connectivity for cloud operations.
- Database persistence and backup must be configured appropriately when deploying on a cloud platform.

APPLICATIONS

- Colleges and universities
- Online learning platforms
- Training and certification centres
- Corporate employee training programs
- Digital document management systems
- Organizations maintaining records of course completion

The system checks file integrity; verification of the actual issuer may require an additional trusted institutional database, digital signature, or issuer-side verification mechanism.

FUTURE ENHANCEMENTS

- Individual user registration and secure login.
- Role-based access for administrators, students, and institutions.
- Digital signatures to verify certificate origin and authenticity.
- QR codes linking to an authorized verification page.
- Integration with official institutional certificate records.
- Improved OCR and automatic field extraction.
- Audit logs, notifications, and detailed reports.
- Database backups and persistent production-grade database storage.
- Stronger access policies and monitoring for cloud resources.

CONCLUSION

The Cloud-Based Digital Certificate Verification & Secure Storage System demonstrates how cloud computing and cryptographic hashing can be combined to manage digital certificates. The application provides certificate uploading, automatic extraction of available details, private cloud storage, encryption at rest, metadata management, and integrity verification.

By comparing SHA-256 hashes, the system can identify when a retrieved certificate file differs from the file recorded during upload. Amazon S3 provides cloud storage, while Flask and SQLite support application processing and metadata management. The project provides a

foundation for a more comprehensive certificate management platform and can be extended with digital signatures, institutional verification, stronger authentication, and audit capabilities.

CLOUD CERTIFICATE VERIFICATION IMPLEMENTATION

SHA-256 Hash Generation (Integrity Module)

```
import hashlib

def generate_sha256(file_bytes):
    return hashlib.sha256(file_bytes).hexdigest()
```

Purpose

Generates a digital fingerprint of the certificate file, which is saved as the expected hash at upload time.

Secure Upload to Amazon S3 (Cloud Storage)

```
import os, boto3

s3 = boto3.client('s3')
BUCKET = os.environ['S3_BUCKET_NAME']

def upload_to_s3(file_bytes, object_key):
    s3.put_object(
        Bucket=BUCKET,
        Key=object_key,
        Body=file_bytes,
        ServerSideEncryption='AES256'
    )
```

Purpose

Stores the certificate in a private S3 bucket with AES-256 server-side encryption enabled.

Certificate Integrity Verification

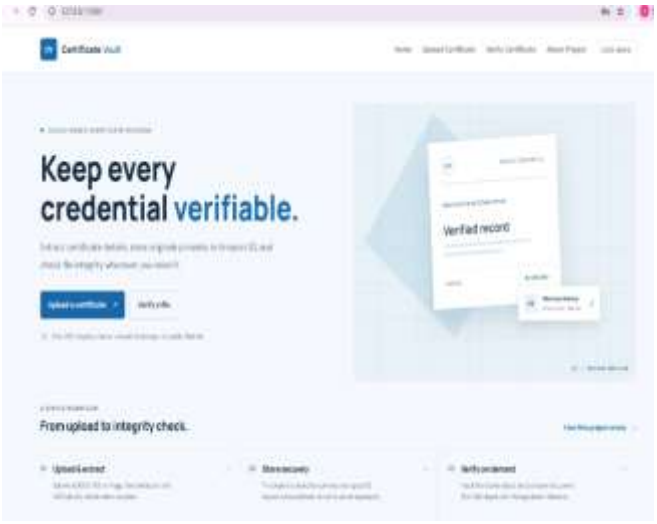
```
def verify_certificate(object_key, expected_hash):
    obj = s3.get_object(Bucket=BUCKET, Key=object_key)
    current_hash = generate_sha256(obj['Body'].read())

    if current_hash == expected_hash:
        return 'Integrity Verified'
    return 'Integrity Mismatch'
```

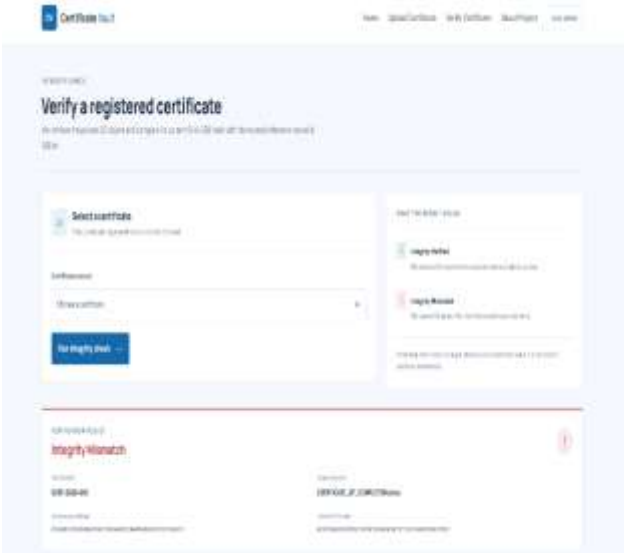
Purpose

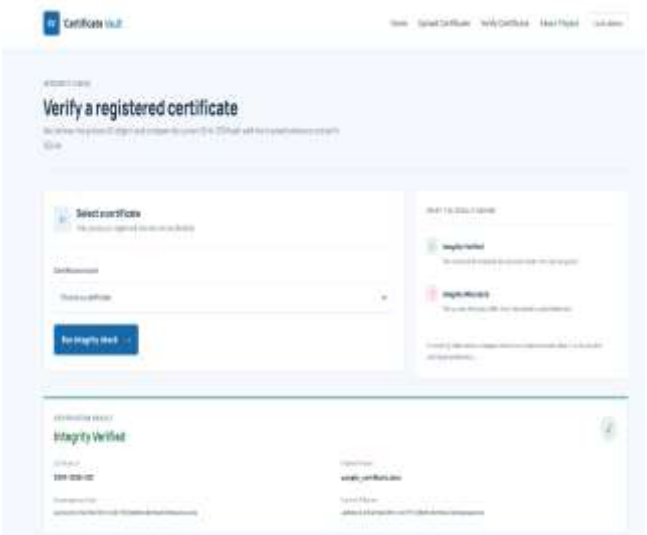
Downloads the stored file from S3, recalculates its hash, and compares it with the expected hash saved in SQLite.

OUTPUT


Keep every credential verifiable.
Verified record
From upload to integrity check.


Certificate registered


Verify a registered certificate
Integrity Mismatch


Verify a registered certificate
Integrity Verified